Container Orchestration with Kubernetes Activities

Table of Contents

Activity 1: Software setup

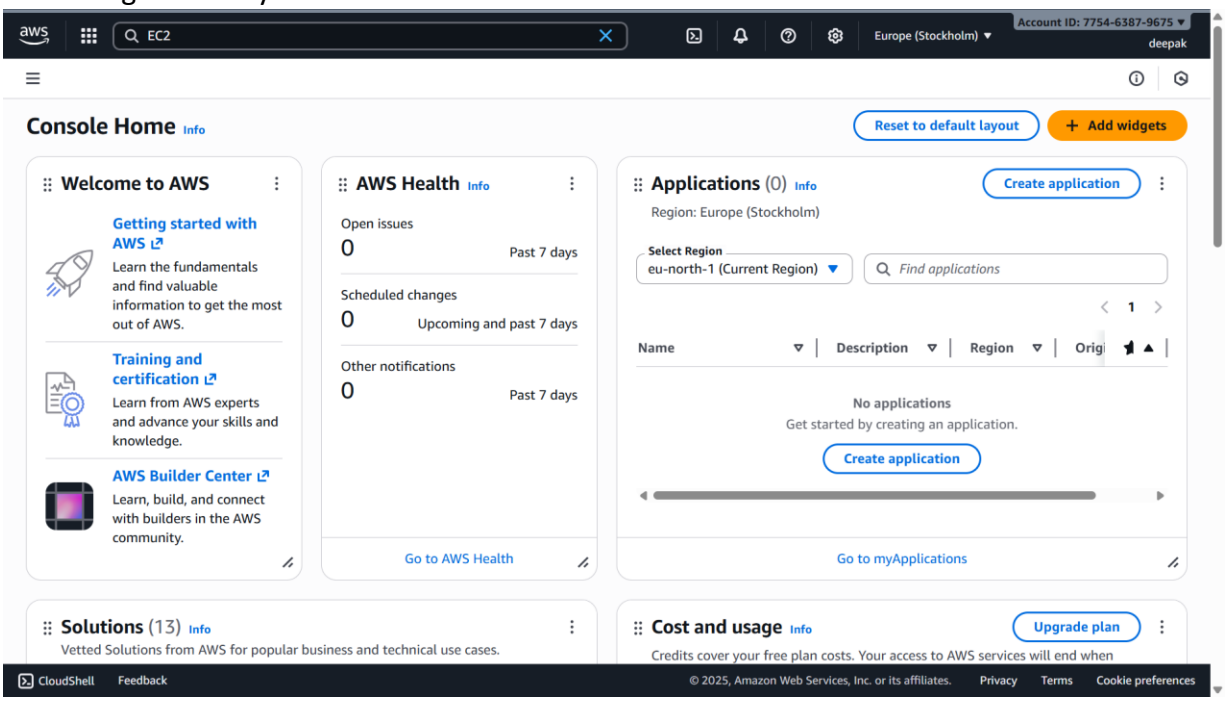
1.1. Prerequisite

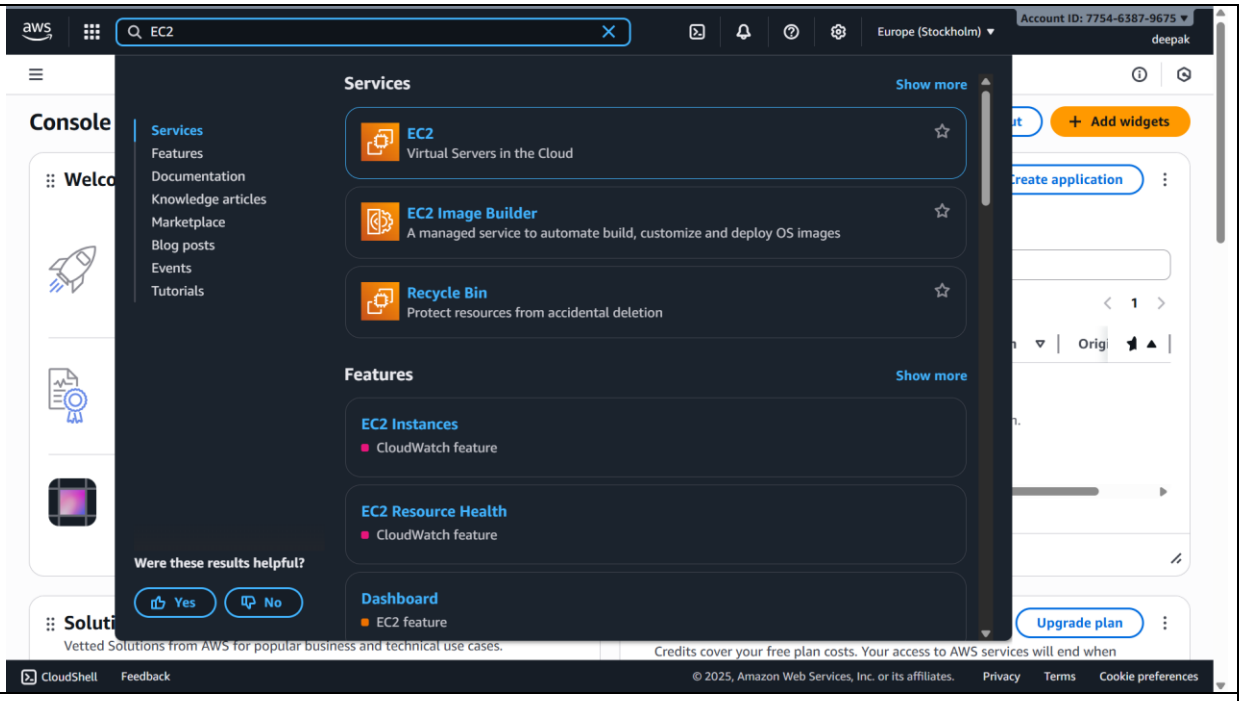
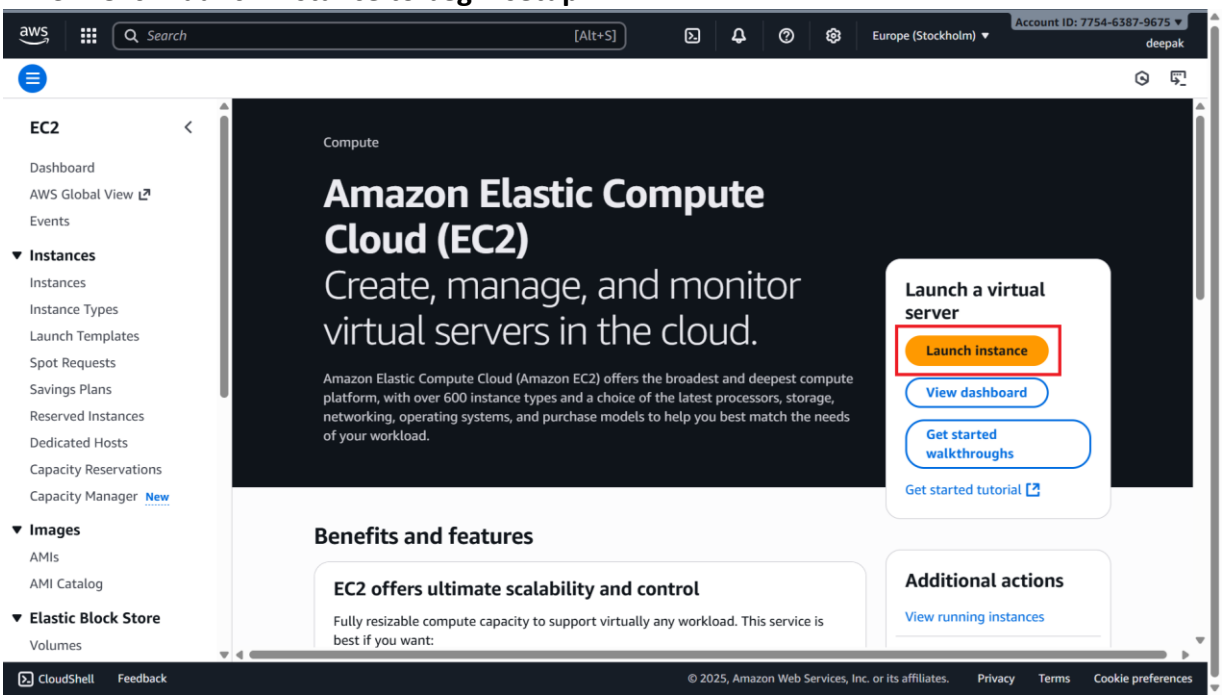
Running virtual machine with ubuntu

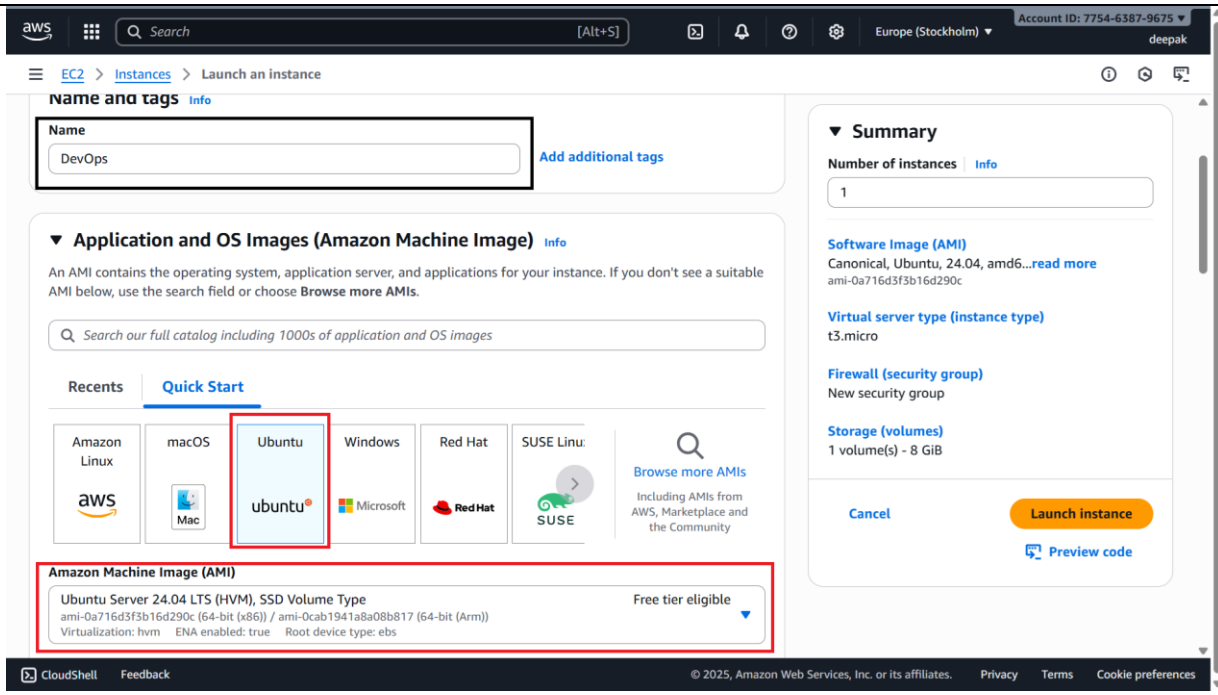
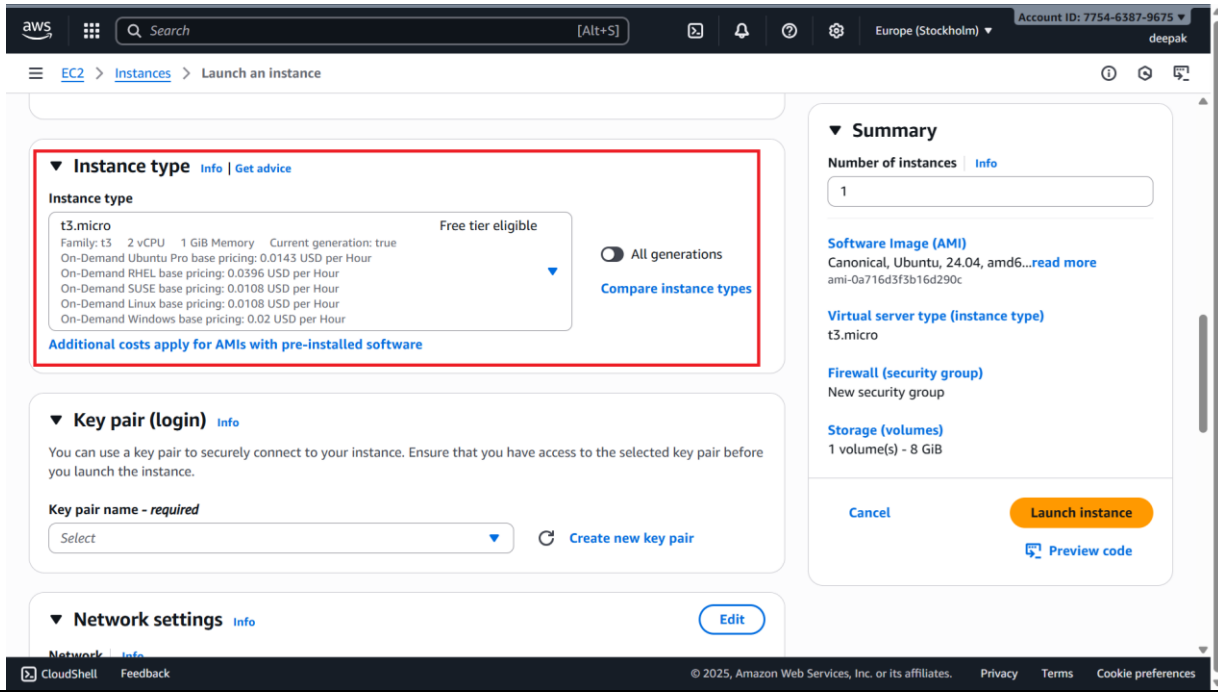
1.2. Scenario

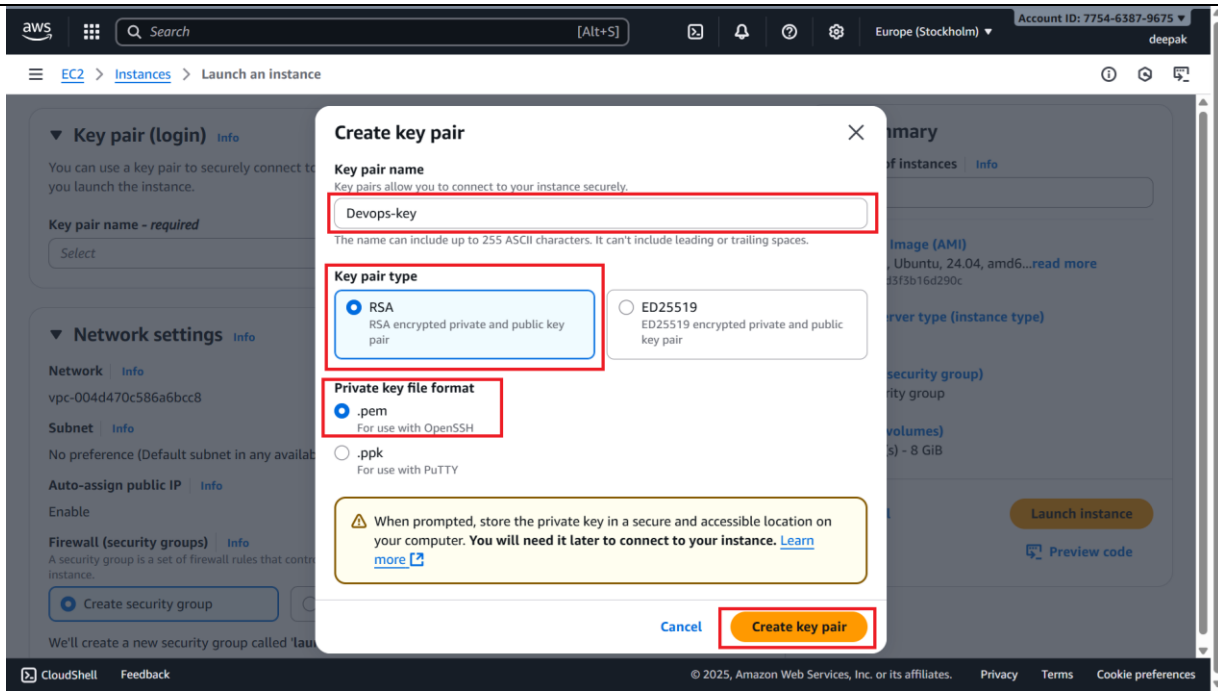
Installing docker, jdk17, maven on ubuntu

1.3. Steps

1	**1. Log in to AWS Console** • Visit https://aws.amazon.com/console • Sign in with your credentials 
2	**2. Navigate to EC2 Dashboard** • In the AWS Console, search for **EC2**

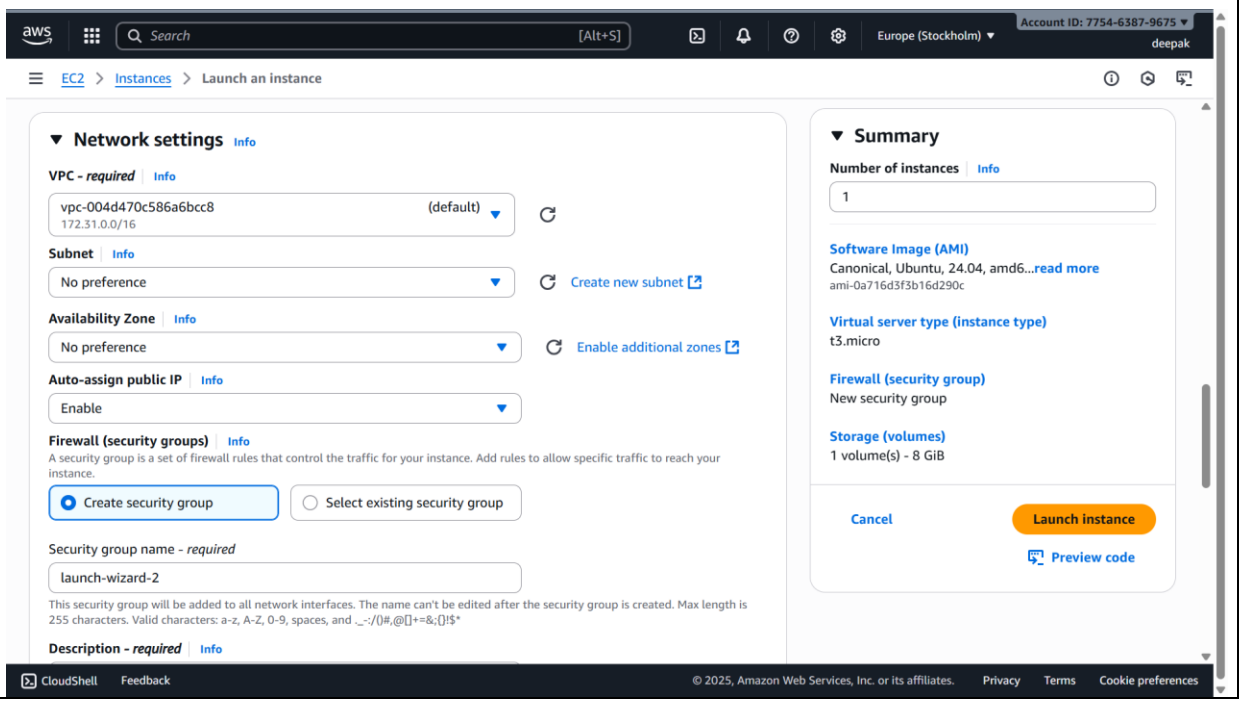
	
3	**3. Click Launch Instance to begin setup** 
4	**3. Choose an Amazon Machine Image (AMI)** • Select **Ubuntu Server 22.04 LTS** or **Ubuntu 24.04 LTS** (recommended for longer support) • Ensure it is marked **Free Tier eligible**

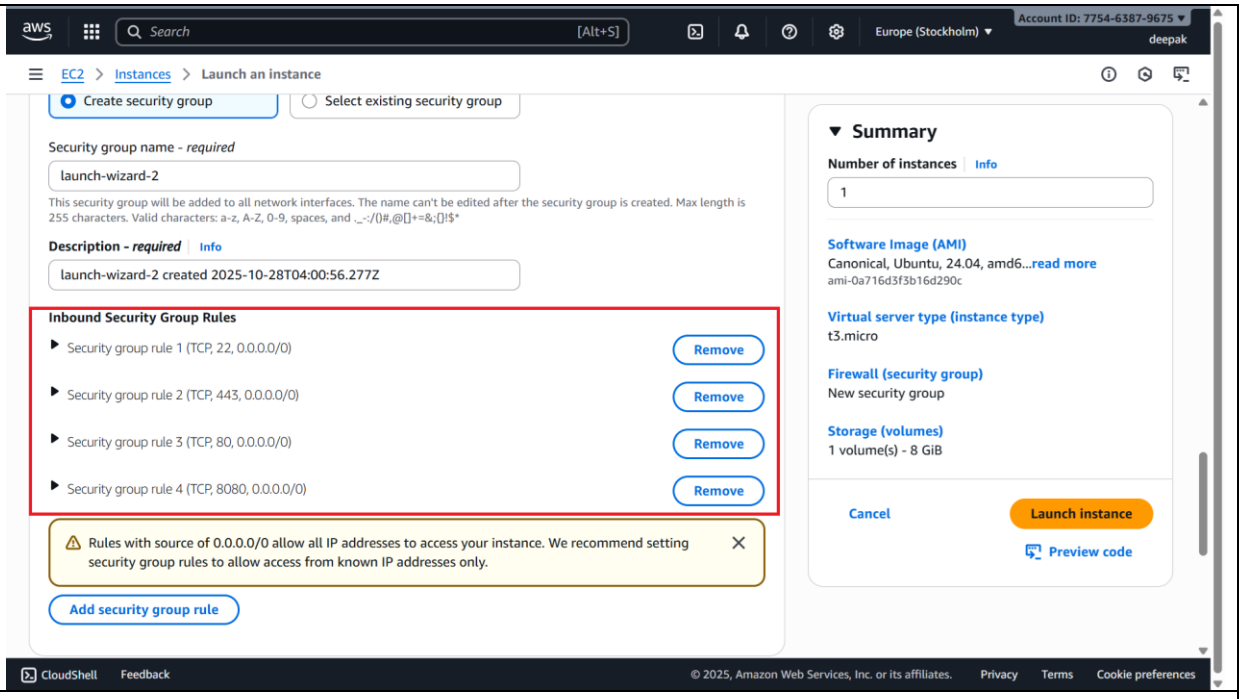
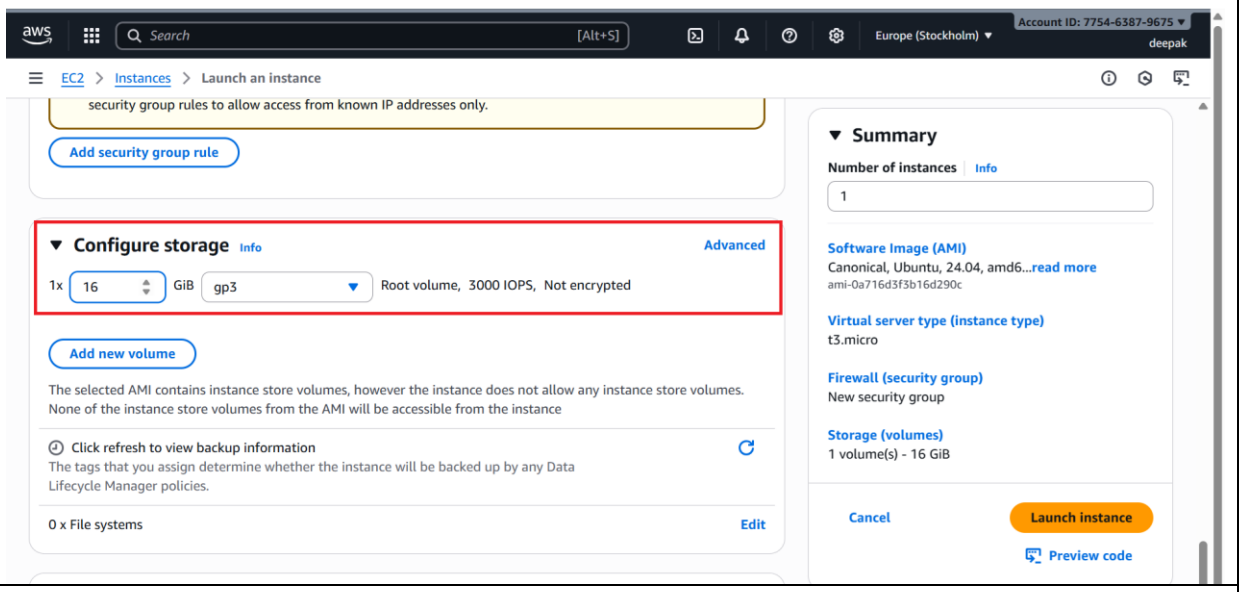
	
5	**4. Select Instance Type** • Choose **t2.micro** (1 vCPU, 1 GB RAM) – Free Tier eligible 
6	**Configure Key Pair** • Create a new key pair or use an existing one • Choose .pem format for Linux/macOS or .ppk for Windows • Download and store securely (you won't be able to download it again)

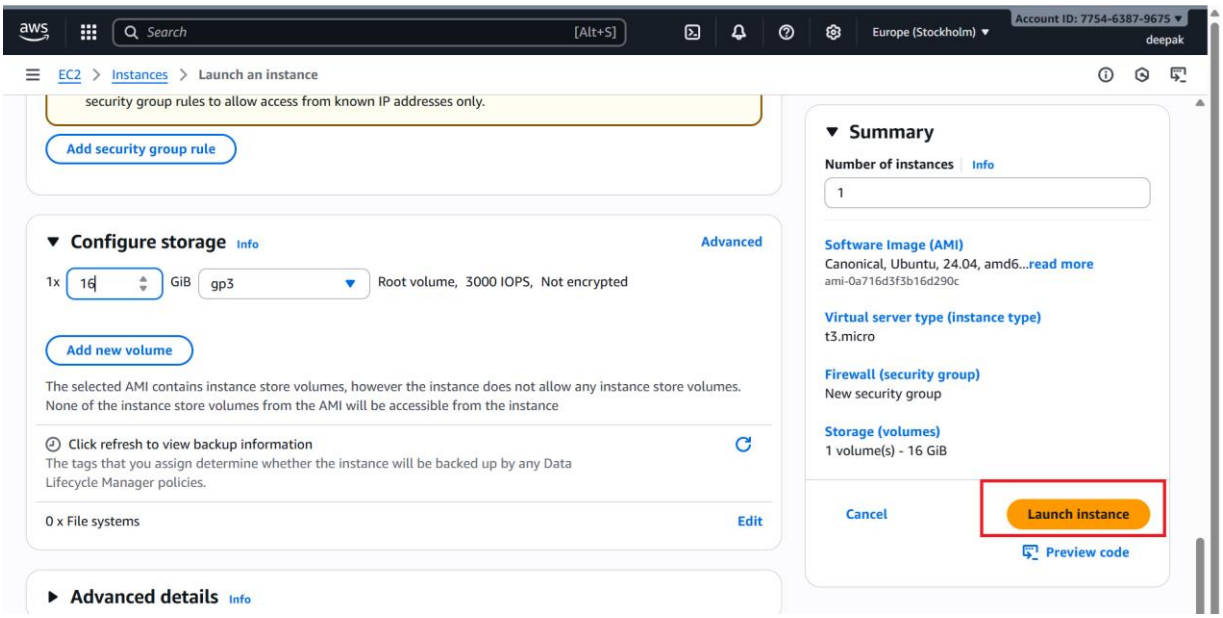


7 **Configure Network Settings**

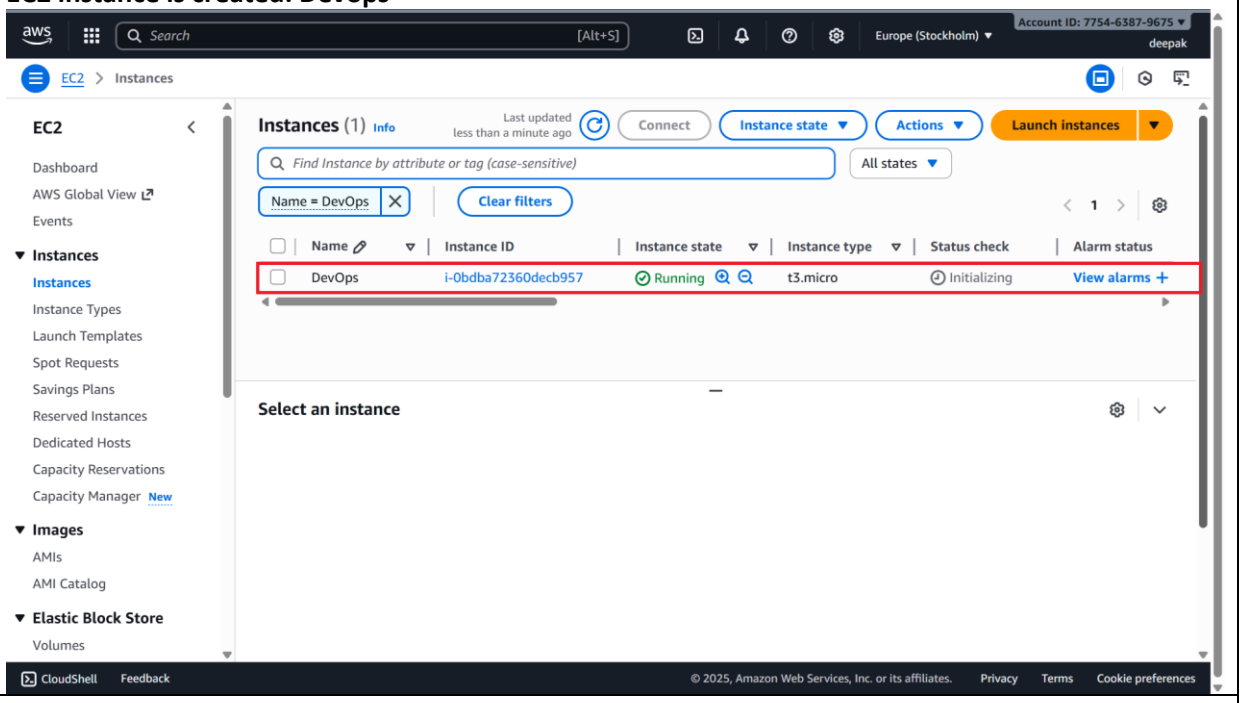
- Use default VPC and subnet
- Create or select a **Security Group**
  - Allow **SSH (port 22)** from your IP
  - Optionally allow **HTTP (port 80)** and **HTTPS (port 443)** if hosting a web server



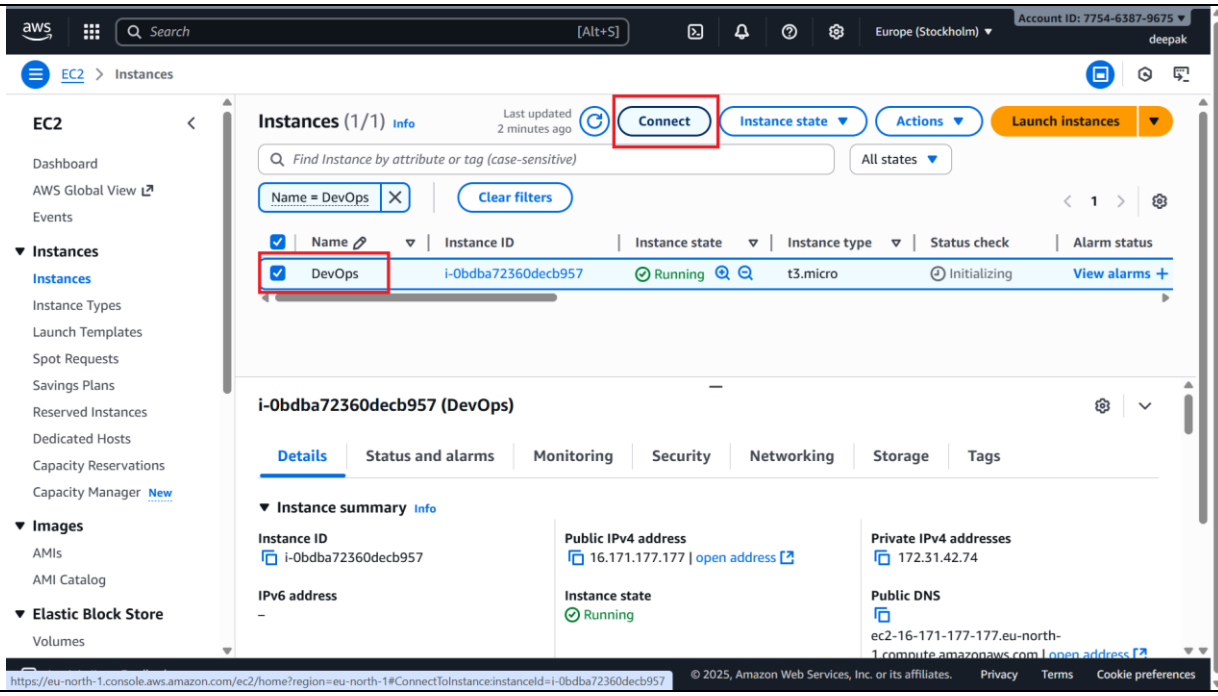
	
8	**7. Configure Storage** • Default is 8 GB SSD (gp3) • Free Tier allows up to **30 GB** of gp3 storage 
9	**Review and Launch** • Review all settings • Click **Launch Instance**



10 **EC2 instance is created: Devops**

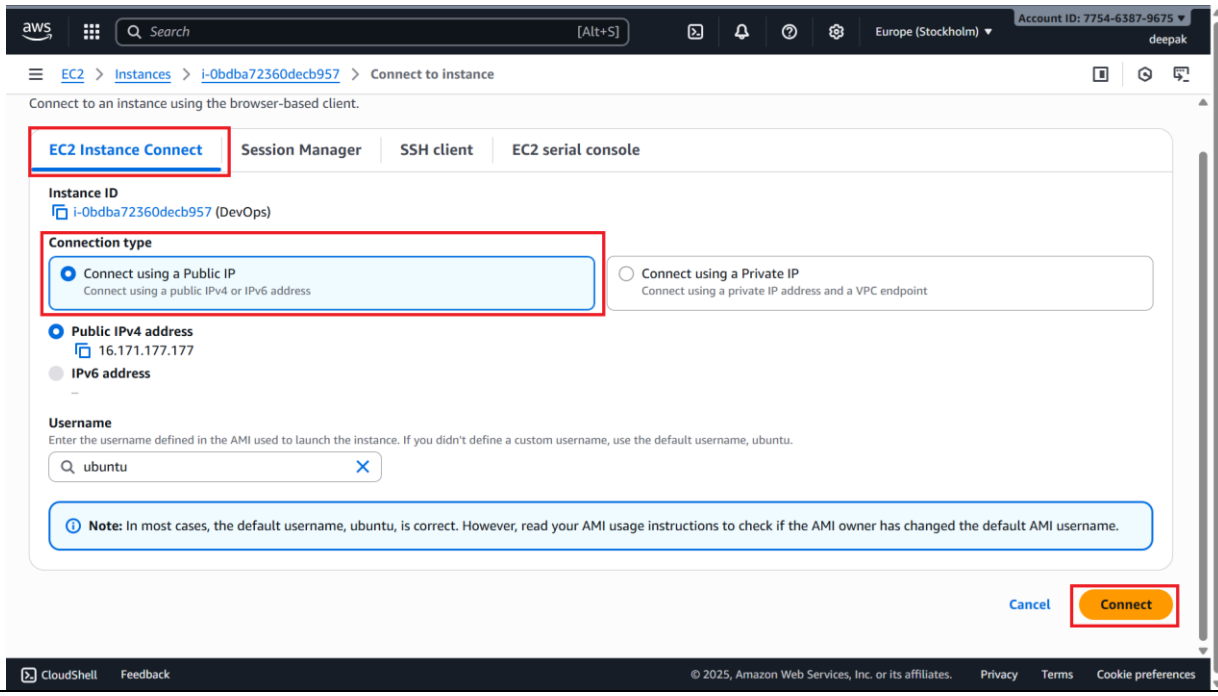


11 **Connect to Your Instance**



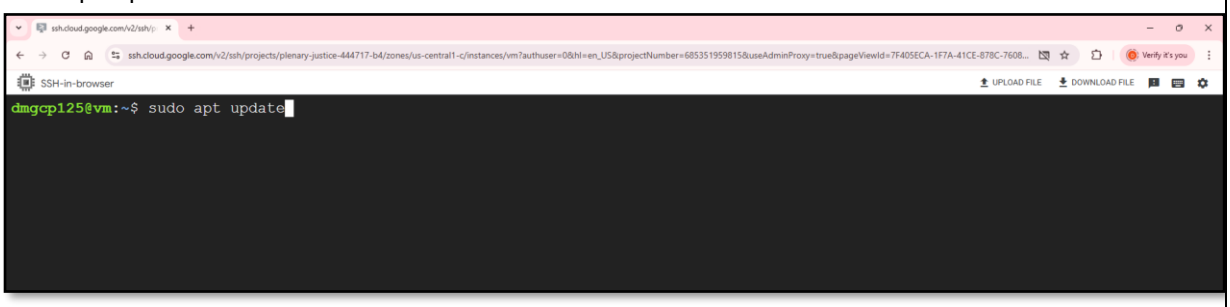
12

Connect to Your Instance



13

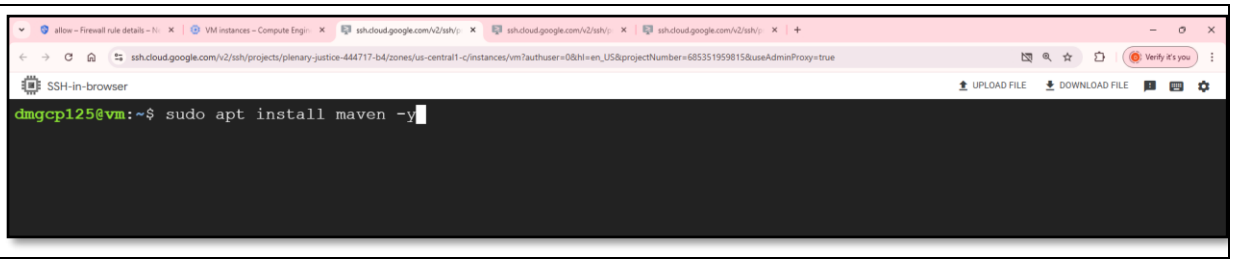
ubuntu packages
sudo apt update



14

Install docker using command: sudo apt install docker.io

	 If prompted press Y
15	Add current user in docker group using command: sudo usermod -aG docker $USER  Reboot machine using command: sudo reboot  Note: Docker group is already created as part of docker installation. Check using command: cat /etc/group. If docker group is not present create using command: sudo newgrp docker
16	Check docker installation: docker --version 
17	Install jdk17 Command: sudo apt install openjdk-17-jdk-headless -y 
18	Install maven Command: sudo apt install maven -y

Activity 2: Setup kubectl

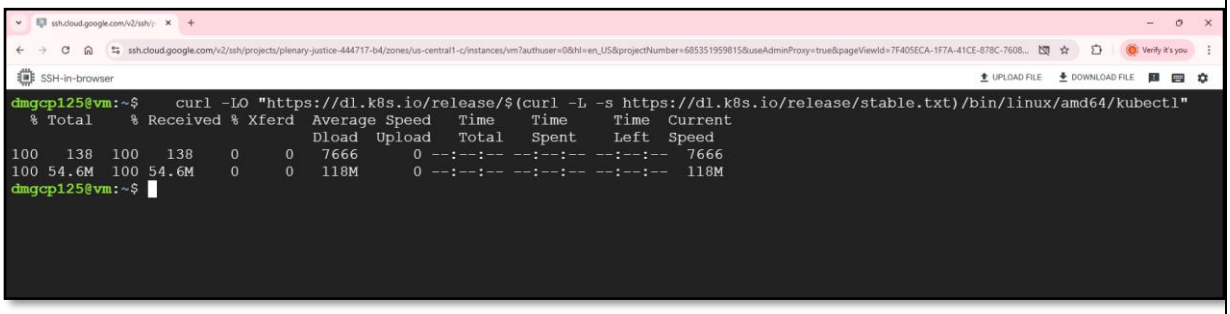
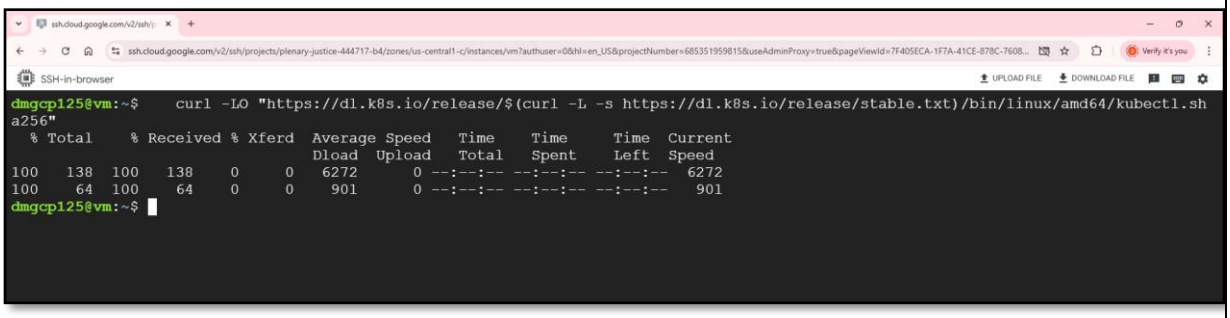
2.1. Prerequisite

Ubuntu VM with docker installed

2.2. Scenario

Installing kubectl

2.3. Steps

1	Go to vm
2	• Download latest release • Command: curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" 
3	• Download kubectl checksum. • Command: curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256" 
4	• Validate kubectl binary. • Command: echo "$(cat kubectl.sha256) kubectl" | sha256sum --check

	
5	• Install kubectl. • Command: sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl 
6	• Check installation • Command: kubectl version --client 

Activity 3: Setup minikube cluster

3.1. Prerequisite

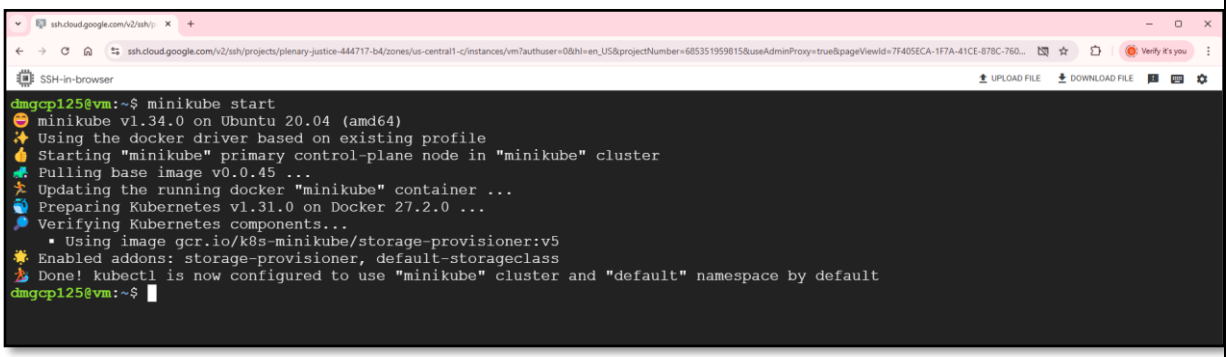
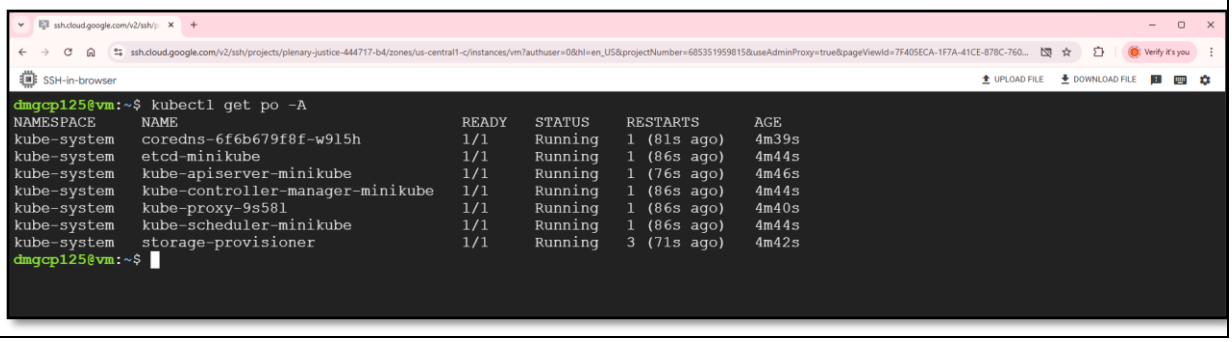
Ubuntu VM with docker, kubectl installed

3.2. Scenario

Installing minikube cluster

3.3. Steps

1	Go to vm
2	• Get latest Debian package • Command: curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64

	
3	• Install minikube • Command: sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64 
4	Start minikube Command: minikube start --driver=docker --force 
5	Output screen: 
6	Execute below command to check installation Command: kubectl get po -A 

Activity 4: Kubernetes commands

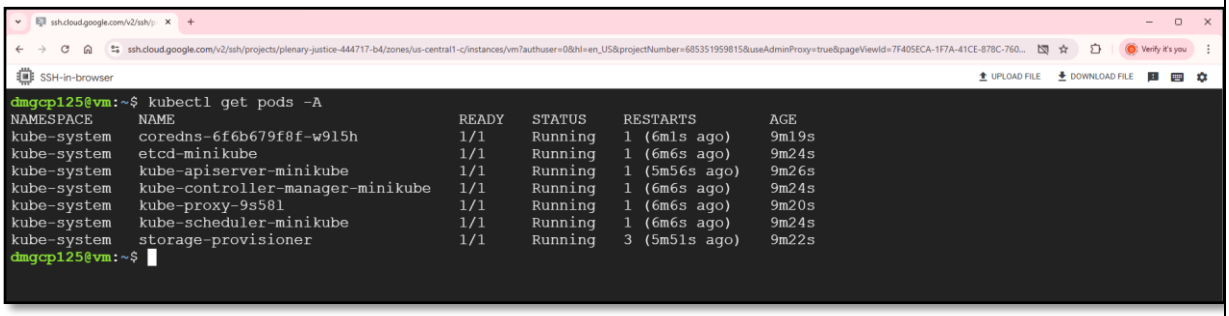
4.1. Prerequisite

Minikube cluster started

4.2. Scenario

Executing Kubernetes commands

4.3. Steps

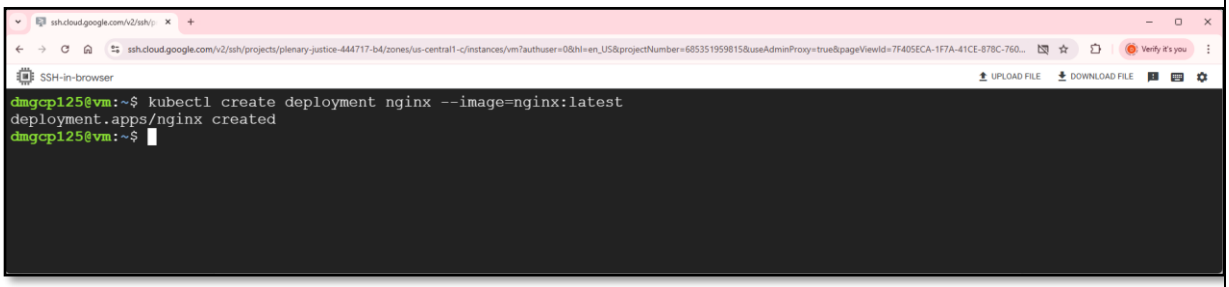
1	Go to vm kubectl version --client (check version kubectl) minikube version minikube delete (execute if any issues) minikube start --driver=docker --force
2	Listing pods: Command: kubectl get pods -A or kubectl get po -A 
3	Create pod using command Command: kubectl run nginx --image=nginx --restart=Never  List pods: Command: kubectl get pods

4 Delete pod

Command: kubectl delete pod nginx

5 Create deployment

Command: kubectl create deployment nginx --image=nginx:latest

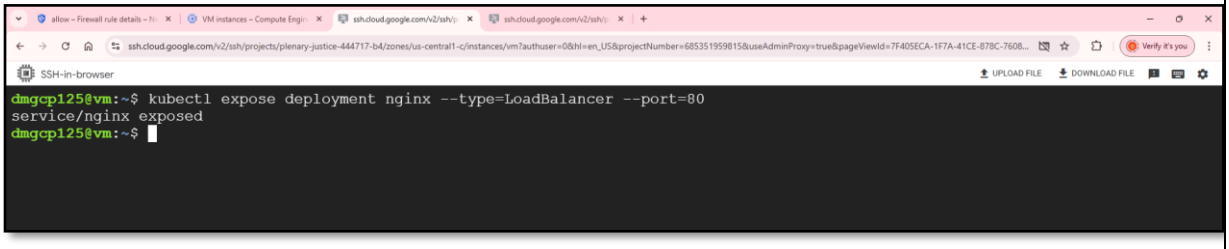


List deployment

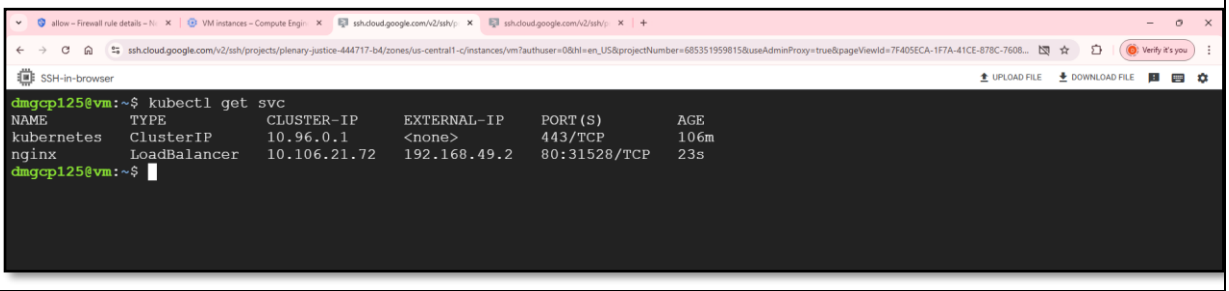
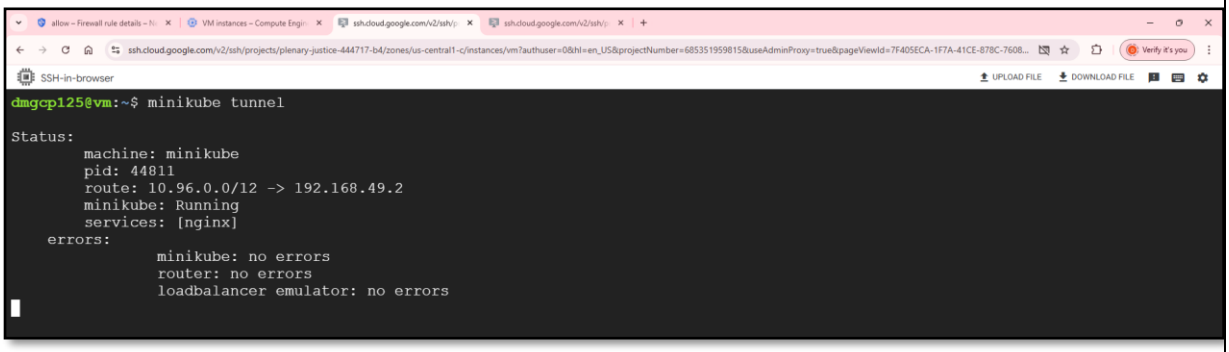
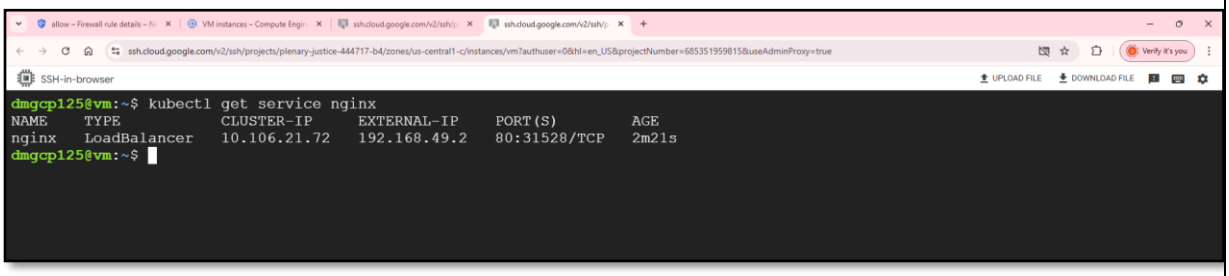
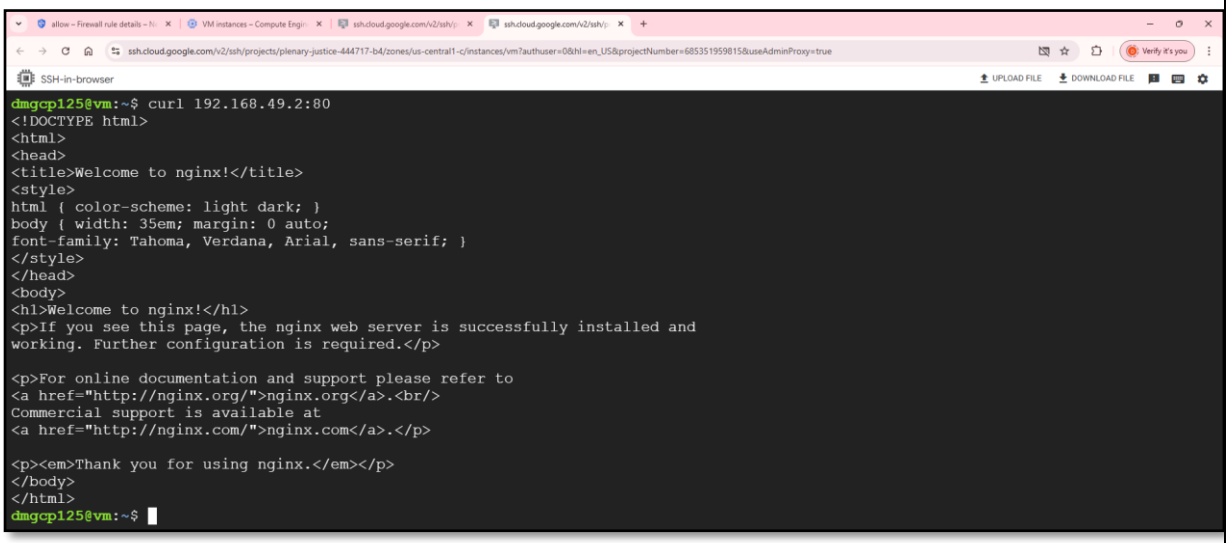
Command: kubectl get deployment

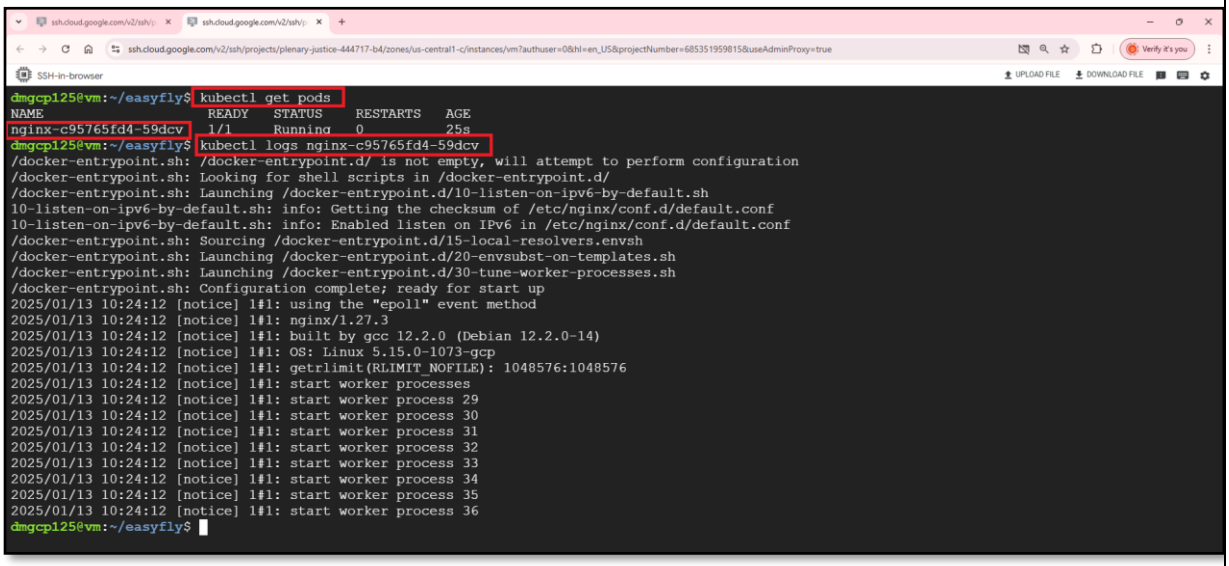
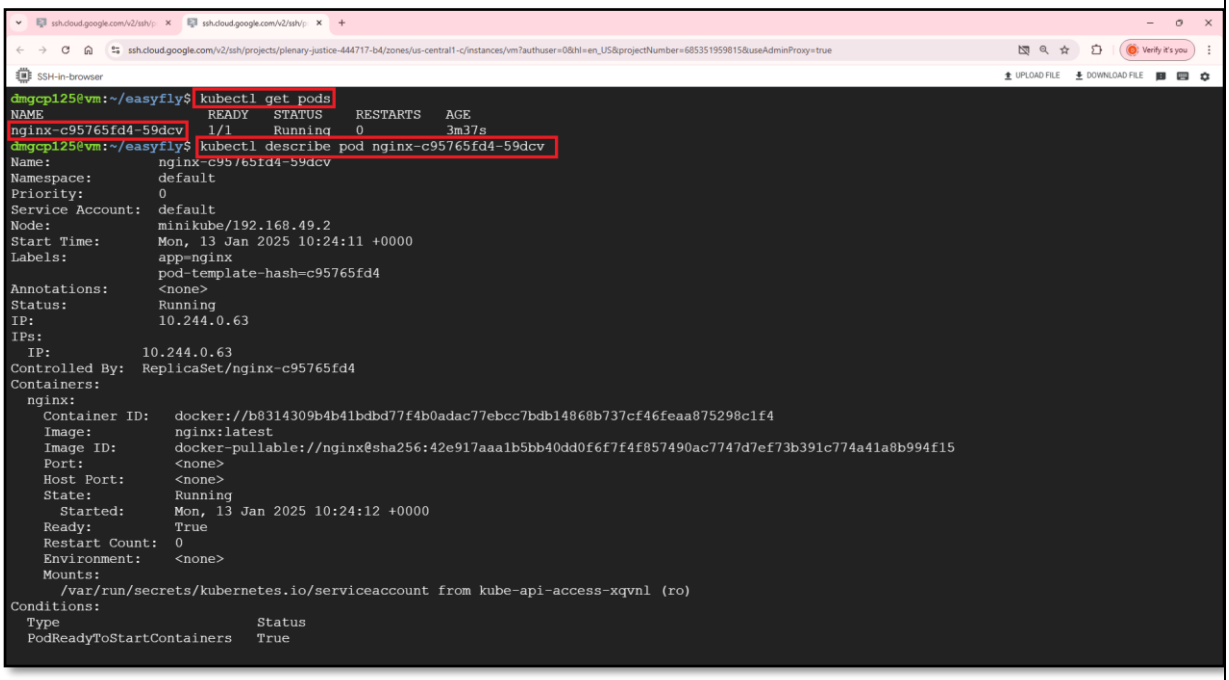
6 Expose deployment

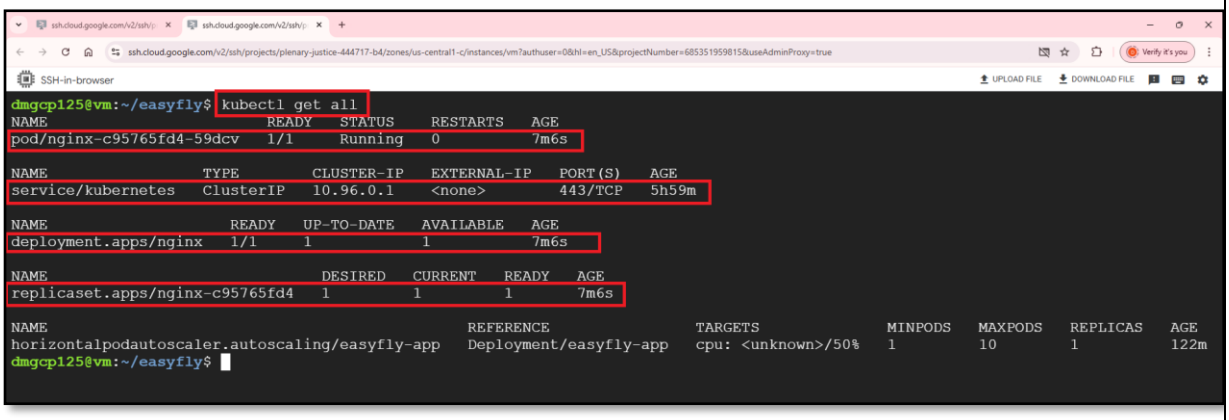
Command: kubectl expose deployment nginx --type=LoadBalancer --port=80



List services

	Command: kubectl get services 
7	To access load balancer service start minikube tunnel Command: minikube tunnel  Open another terminal .Get ip address : Command: kubectl get service nginx  Access application with curl command: curl 192.168.49.2:80 (replace ip with external-ip from previous step) 
8	Check logs of pod Get pd name using command; kubectl get pods

	Use same name in below command Command: kubectl logs <pod-name> 
9	Get pod in-depth details Command: kubectl get pods -o wide 
10	Explain pod Command: kubectl describe pod <pod-name> 
11	Get replicaset Command: kubectl get rs

	
12	Get all resources Command: kubectl get all minikube dashboard 
13	Get deployment in yml format Command: kubectl get deployment nginx -o yaml >> deployment.yaml cat deployment.yaml apiVersion: apps/v1 kind: Deployment metadata: annotations: deployment.kubernetes.io/revision: "1" creationTimestamp: "2025-01-13T10:24:11Z" generation: 1 labels: app: nginx name: nginx namespace: default resourceVersion: "40096" uid: 97b2cf77-f6ec-4645-a0d8-5405c4ea2d85 spec: progressDeadlineSeconds: 600 replicas: 1 revisionHistoryLimit: 10 selector: ….
14	Create pod using yaml file vi pod.yml

	after Entering below content first press the Esc key to enter command mode :wq (to save and quit), and press Enter. apiVersion: v1 kind: Pod metadata: name: nginx-pod-using-yaml-file labels: app: nginx spec: containers: - name: nginx image: nginx ports: - containerPort: 80 Create pod Command: kubectl apply -f pod.yml

Activity 5: Deploy Spring Boot Application

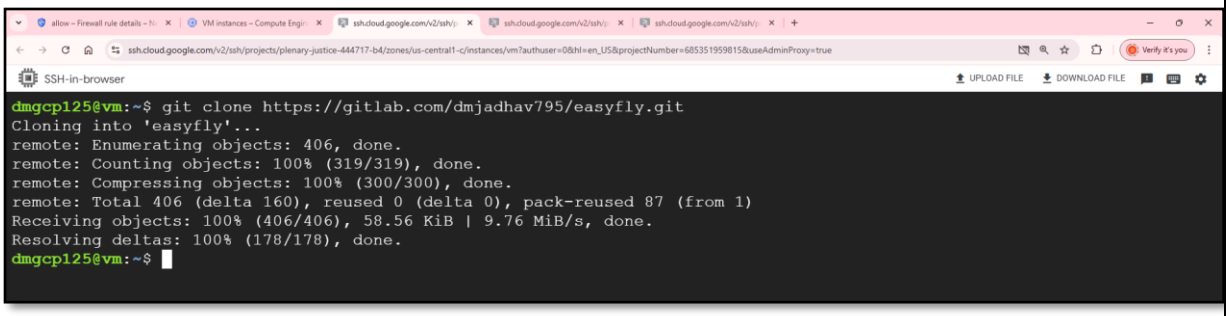
5.1. Prerequisite

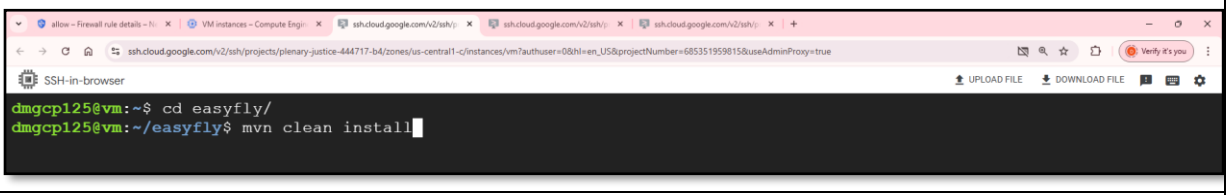
Minikube cluster started.

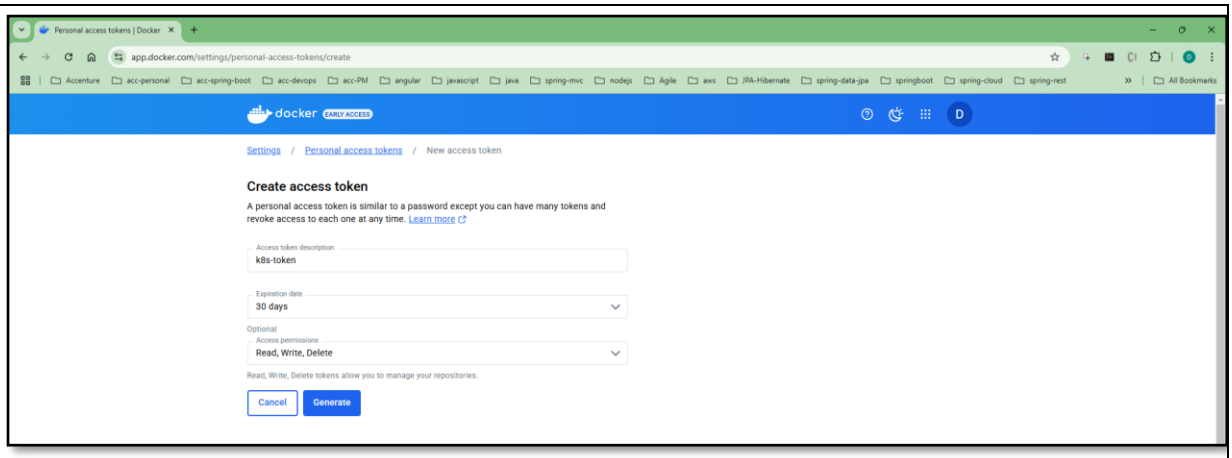
5.2. Scenario

Deploying spring boot application in minikube cluster

5.3. Steps

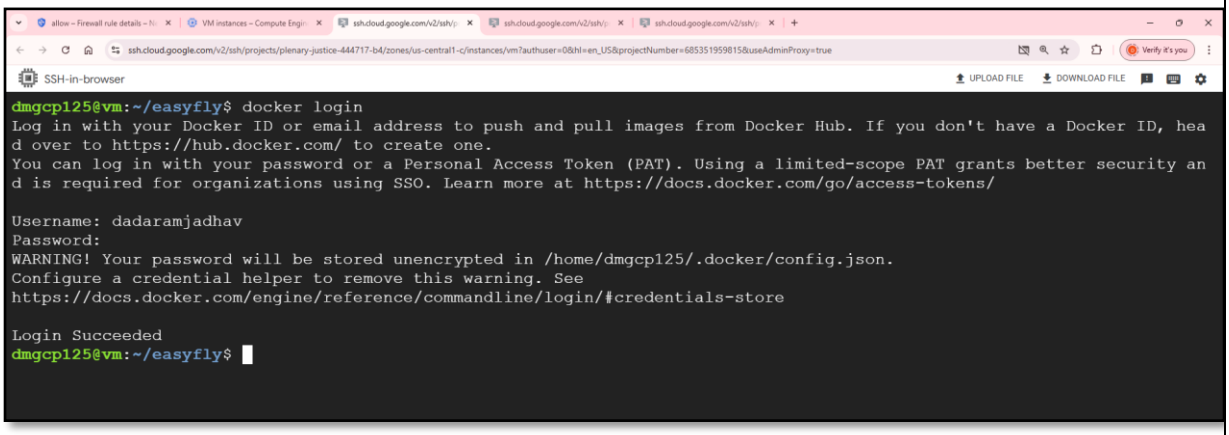
1	Go to vm Login to GitLab account Fork Repository https://gitlab.com/dadaramjadhav795/easyfly.git
2	Remove easyfly application if present rm -r easyfly clone the repository git clone https://gitlab.com/ dadaramjadhav795/easyfly.git 
3	Build application Command: cd easyfly

	mvn clean install ls target 
4	Create dockerfile in root directory. touch Dockerfile-dev vim Dockerfile-dev To insert text, press `i`. After typing your content, press `Esc` to exit insert mode. If you want to save and exit at the same time, type `:wq` and press `Enter`. cat Dockerfile-dev #this is dockerfile for development #use Dockerfile-dev and edit it FROM --platform=linux/arm64 eclipse-temurin:17-jdk COPY target/*.jar app.jar ENV server.port=8081 ENTRYPOINT ["java", "-Dspring.profiles.active=dev", "-jar","/app.jar"]
5	Login to Docker Hub docker login enter usrname of Docker Hub account and enter pass code Build docker image Command: docker build -t dadaramjadhav/easyfly:dev -f Dockerfile-dev 
6	Create personal access token in dockerhub a. Login into docker hub b. Go to https://app.docker.com/settings/personal-access-tokens c. Click on generate new token d. Enter token name e. Choose expiration time f. Choose access permission (Read, Write, Delete) g. Click on generate



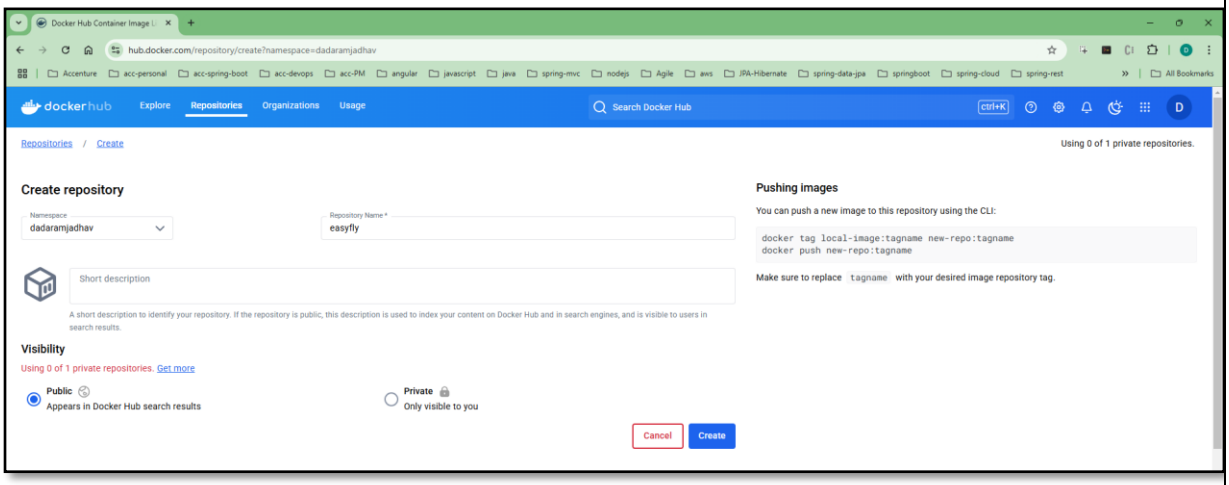
h. Copy personal access token generated

7 Login into docker using command docker login
Username: Enter dockerhub username
Password: enter personal access token (created in previous step)

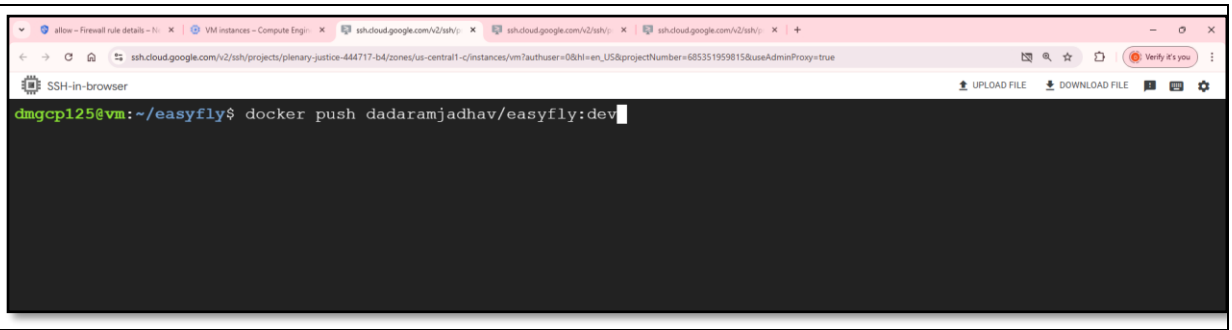
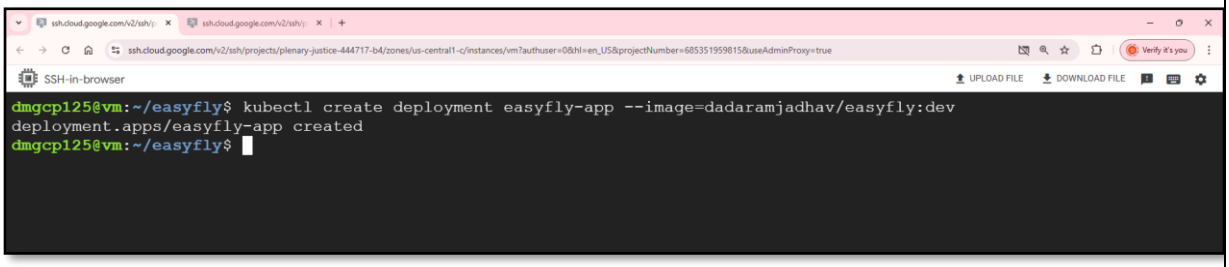
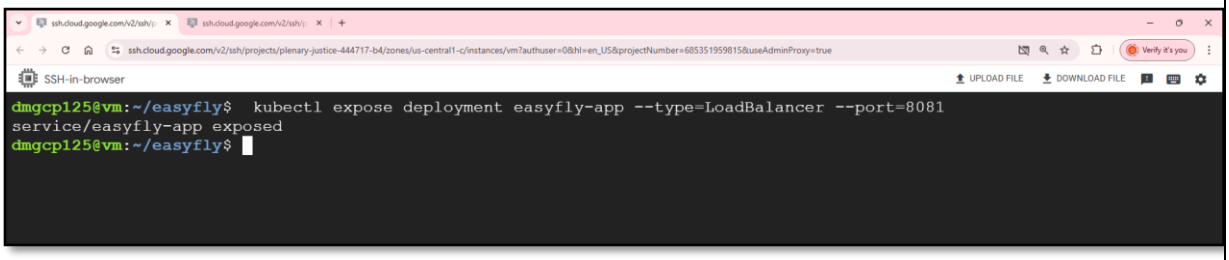


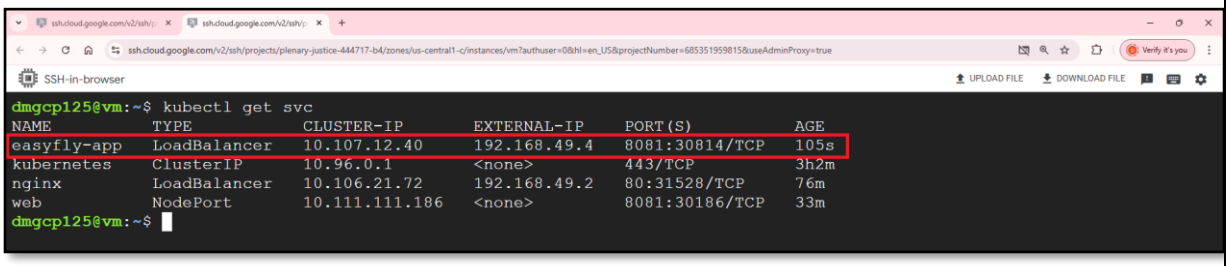
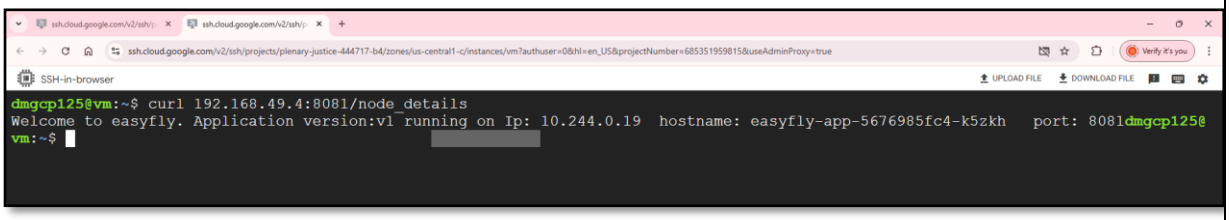
8 Create registry into docker hub

a. Go to https://hub.docker.com/
b. Click on create repository
c. Enter repository name and click on create



9 Push image to docker hub registry
Check list of images: docker images
Command: docker push dadaramjadhav795/easyfly:dev

	
10	Create deployment Command: kubectl create deployment easyfly-app --image= dadaramjadhav/easyfly:dev 
11	Expose deployment Command: kubectl expose deployment easyfly-app --type=LoadBalancer --port=8081 check deployments kubectl get deployments check pods kubectl get pods check log of created pod kubect logs <pod-id> 
12	Start minikube tunnel to access loadbalancer service using command minikube tunnel  Keep this terminal running
13	Open another terminal List services

	Command: kubectl get svc 
14	Access application Command: curl 192.168.49.4:8081/node_details  **Other endpoints** 1. curl 192.168.49.4:8081/api-v1/flights/ (GET) 2. curl 192.168.49.4:8081/api-v1/flights/3 (GET) 3. curl --header "Content-Type: application/json" --request POST --data '{ "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/ (POST Request) 4. curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6, "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/6 (PUT request) 5. curl --request DELETE 192.168.49.4:8081/api-v1/flights/6 (DELETE)

Activity 6: Scale spring boot application

6.1. Prerequisite

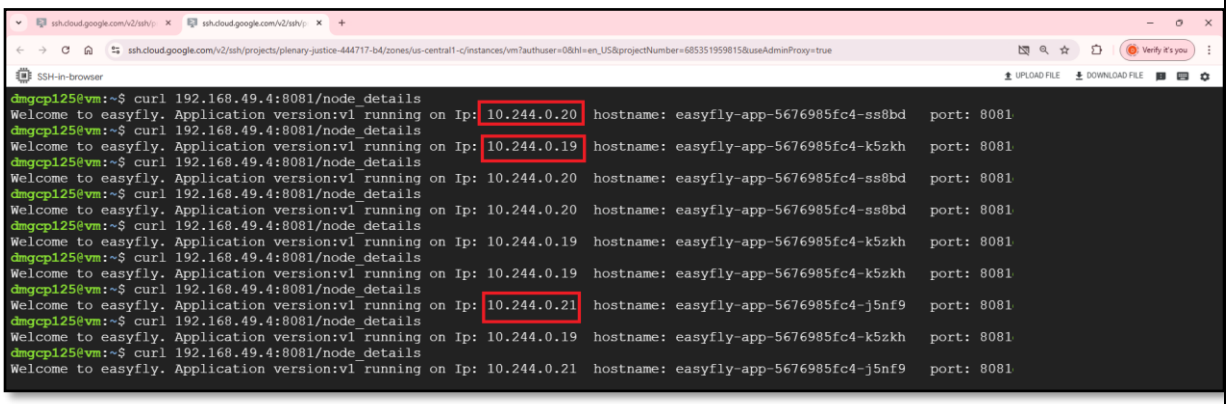
Spring boot application running inside minikube cluster

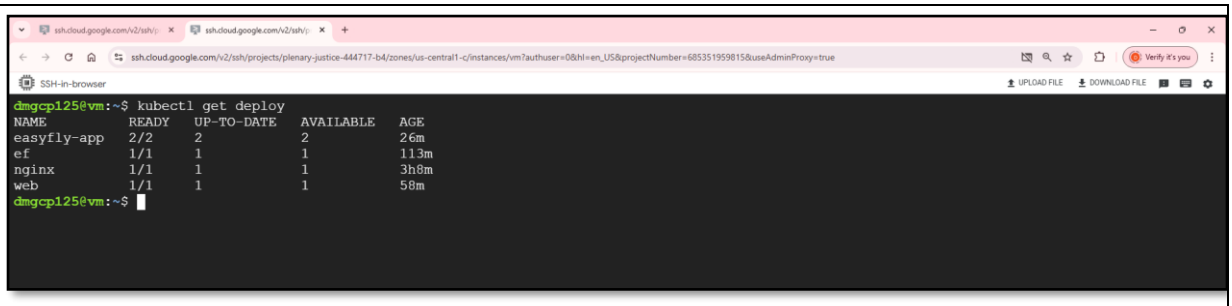
6.2. Scenario

Scaling spring boot application to handle increasing load

6.3. Steps

1	Go to vm
2	Scale deployment

	Command: kubectl scale deploy easyfly-app --replicas=3  Check deployment Command: kubectl get deploy check pods kubectl get pods 
3	Access application Run application for 5-6 times. You will get response from different container. Command: curl 192.168.49.4:8081/node_details 
4	Scale down application Command: kubectl scale deploy easyfly-app --replicas=2  Get deployment Command: kubectl get deploy

	 Keep hitting this endpoint to confirm scaling curl 192.168.49.4:8081/node_details
5	Scale up application Command: kubectl scale deploy easyfly-app --replicas=5  List deployment Command: kubectl get deploy  Keep hitting this endpoint to confirm scaling curl 192.168.49.4:8081/node_details

Activity 7: Autoscaling spring boot application

7.1. Prerequisite

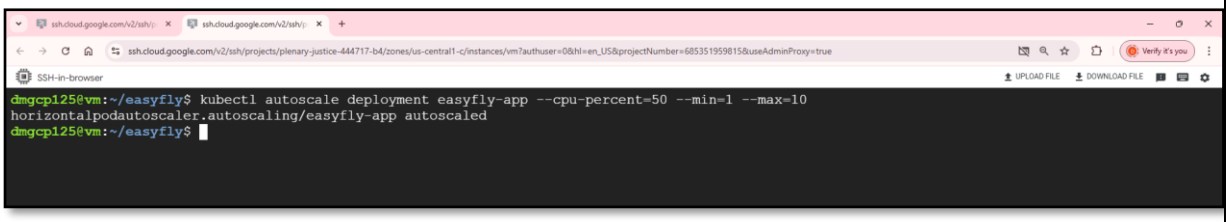
Spring boot application running

7.2. Scenario

Autoscaling spring boot application to handle dynamic load

7.3. Steps

1	Go to vm
2.	Autoscale application based on CPU utilization

	Command: kubectl autoscale deployment easyfly-app --cpu-percent=50 --min=1 --max=10  • This defines a Horizontal Autoscaler that maintains between 1 and 10 replicas of the Pods controlled by the easyfly-app deployment. • It will increase and decrease the number of replicas (via the deployment) so as to maintain an average CPU utilization across all Pods to 50%. • Check status of auto scalar with • Command: kubectl get hpa 

Activity 8: Deploying Spring Boot Application using manifest file

8.1. Prerequisite

Spring boot application

8.2. Scenario

Deploy spring boot application using manifest files

8.3. Steps

1	Go to vm
2.	Remove easyfly directory if present Rm -r easyfly Clone source code Command: git clone https://gitlab.com/ dadaramjadhav795/easyfly.git Change directory cd easyfly
3	Build application Command: mvn clean install

4

touch Dockerfile-dev
vim Dockerfile-dev
To insert text, press `i`. After typing your content, press `Esc` to exit insert mode. If you want to save and exit at the same time, type `:wq` and press `Enter`.
cat Dockerfile-dev

```
#this is dockerfile for development
#use Dockerfile-dev and edit it

FROM --platform=linux/arm64 eclipse-temurin:17-jdk

COPY target/*.jar app.jar

ENV server.port=8081

ENTRYPOINT ["java",  "-Dspring.profiles.active=dev", "-jar","/app.jar"]
```

Build docker image
Command: docker build -t dadaramjadhav/easyfly -f Dockerfile-dev

5

Push docker image to docker hub
Command: docker push dadaramjadhav/easyfly:latest

6

Deploy application using below manifest file
Use command: vim deployment.yml
To insert text, press `i`. After typing your content, press `Esc` to exit insert mode. If you want to save and exit at the same time, type `:wq` and press `Enter`.
cat deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: easyfly
spec:
  selector:
    matchLabels:
      app: easyfly
  replicas: 3
  template:
```

```
  metadata:
    labels:
      app: easyfly
  spec:
    containers:
      - name: easyfly
        image: dadaramjadhav/easyfly:dev
        ports:
          - containerPort: 8081
---

apiVersion: v1
kind: Service
metadata:
 name: easyfly-svc
spec:
 selector:
  app: easyfly
 ports:
  - protocol: "TCP"
    port: 8081
    targetPort:
 type: LoadBalancer
```

Deployment
Command: kubectl apply -f deployment.yml

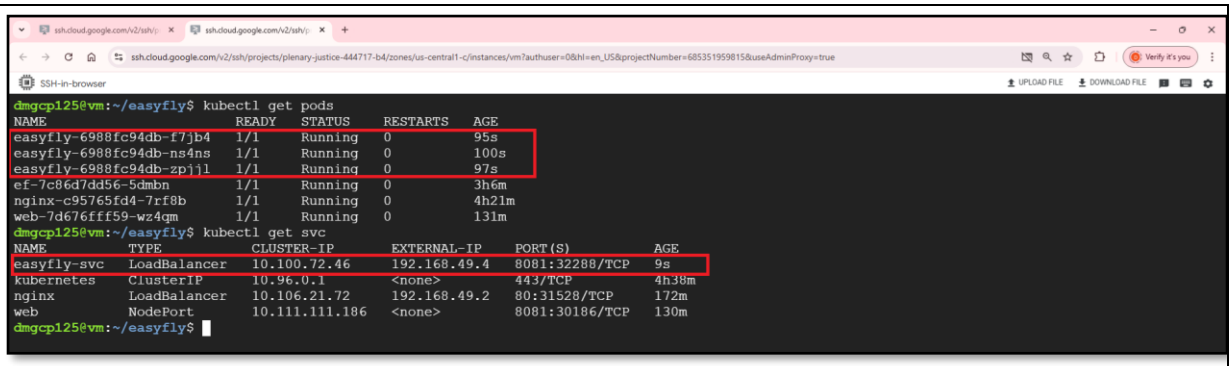
check deployment
kubectl get deployments

check pods
kubectl get pods

check services
kubectl get svc

start minikube tunnel
minikube tunnel

open new terminal



7 Access application using loadbalacer ip

Command: curl 192.168.49.4:8081/node_details

Try Other endpoints

1. curl 192.168.49.4:8081/api-v1/flights/ (GET)
2. curl 192.168.49.4:8081/api-v1/flights/3 (GET)
3. curl --header "Content-Type: application/json" --request POST --data '{ "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/ (POST Request)
4. curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6, "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/6 (PUT request)
5. curl --request DELETE 192.168.49.4:8081/api-v1/flights/6 (DELETE)

8 Scaling application

Update deployment.yml (update replica to 5)

vim deployment.yml

cat deployment.yml

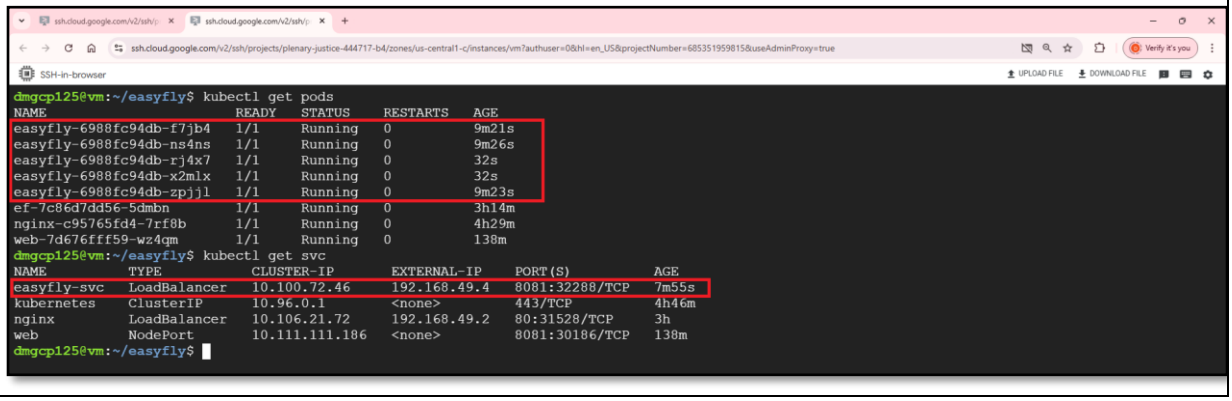
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: easyfly
spec:
  selector:
    matchLabels:
      app: easyfly
  replicas: 5
  template:
```

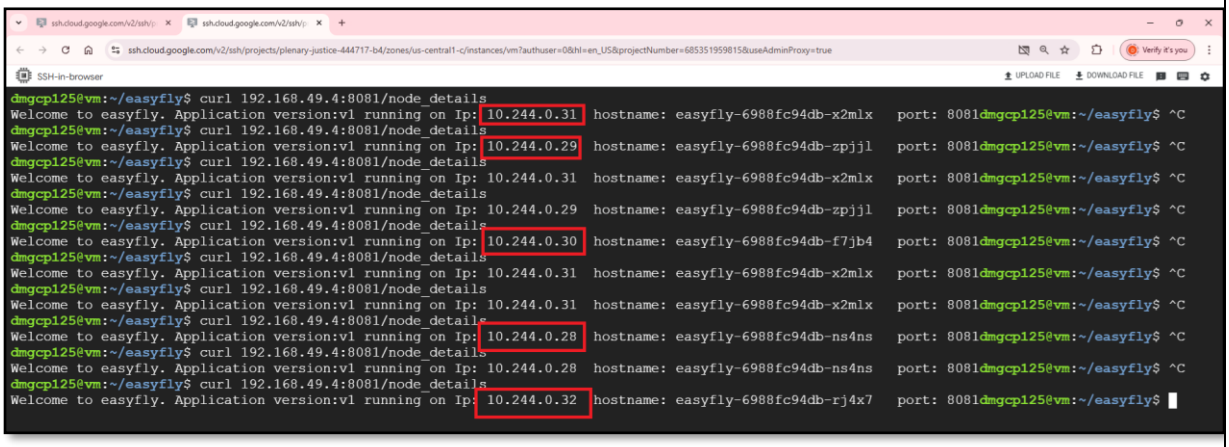
```
  metadata:
    labels:
      app: easyfly
  spec:
    containers:
      - name: easyfly
        image: deepakmoud/easyfly:dev
        ports:
          - containerPort: 8081
---

apiVersion: v1
kind: Service
metadata:
 name: easyfly-svc
spec:
 selector:
  app: easyfly
 ports:
  - protocol: "TCP"
    port: 8081
    targetPort:
 type: LoadBalancer
```

Apply deployment
Command: kubectl apply -f deployment.yml

Check deployment
Command:
kubectl get pods
kubectl get svc



9	Access scaled application Command: curl 192.168.49.4:8081/node_details  **Try Other endpoints** 1. curl 192.168.49.4:8081/api-v1/flights/ (GET) 2. curl 192.168.49.4:8081/api-v1/flights/3 (GET) 3. curl --header "Content-Type: application/json" --request POST --data '{ "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/ (POST Request) 4. curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6, "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v1/flights/6 (PUT request) 5. curl --request DELETE 192.168.49.4:8081/api-v1/flights/6 (DELETE)

Activity 9: Changing Deployment Image

9.1. Prerequisite

Spring boot application

9.2. Scenario

Changing deployment image for spring boot application using manifest files

9.3. Steps

1	Go to vm
2	Update source code to api-v2
3	Pull updated source code Command: git pull
4	Build application

	
5	Build docker image Command: docker build -t dadaramjadhav/easyfly:dev-v2 -f Dockerfile-dev . 
6	Push image into docker hub Command: docker push dadaramjadhav/easyfly:dev-v2 
7	Updated deployment.yml apiVersion: apps/v1 kind: Deployment metadata: name: easyfly spec: selector: matchLabels: app: easyfly replicas: 5 template: metadata: labels: app: easyfly spec: containers: - name: easyfly image: dadaramjadhav/easyfly:dev-v2 ports: - containerPort: 8081 --- apiVersion: v1 kind: Service metadata: name: easyfly-svc

```
spec:
  selector:
    app: easyfly
  ports:
    - protocol: "TCP"
      port: 8081
      targetPort:
  type: LoadBalancer
```

Update deployment
Command: kubectl apply -f deployment.yml

8

Access application
Access scaled application
Command: curl 192.168.49.4:8081/node_details

Try Other endpoints

1. curl 192.168.49.4:8081/api-v2/flights/ (GET)
2. curl 192.168.49.4:8081/api-v2/flights/3 (GET)
3. curl --header "Content-Type: application/json" --request POST --data '{ "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v2/flights/
   (POST Request)
4. curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6, "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v2/flights/6
   (PUT request)
5. curl --request DELETE 192.168.49.4:8081/api-v2/flights/6 (DELETE)

Activity 10: Deploying Spring Boot App with MySQL

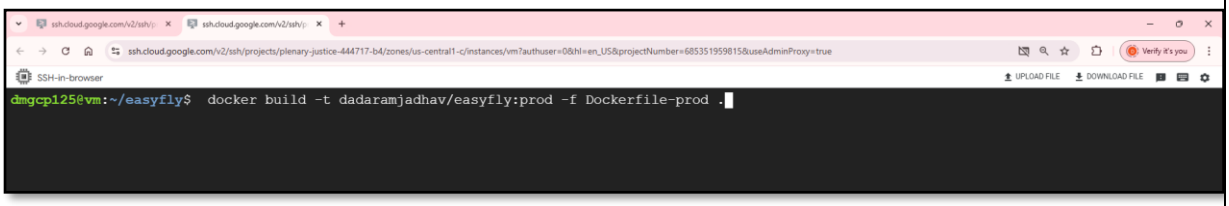
10.1. Prerequisite

Spring boot application with MySQL

10.2. Scenario

Deploying spring boot application with MySQL using manifest files

10.3. Steps

1	Go to vm
2	Build application Command: mvn clean install 
3	Build docker image Command: docker build -t dadaramjadhav/easyfly:prod -f Dockerfile-prod .  Push docker image Command: docker push dadaramjadhav/easyfly:prod 
4	Add db-deployment.yml file. It will create docker container for MySQL apiVersion: v1 kind: PersistentVolumeClaim metadata: name: mysql-pv-claim labels: app: mysql tier: database spec:

```
  accessModes:
   - ReadWriteOnce
  resources:
   requests:
    storage: 1Gi
---

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  labels:
   app: mysql
   tier: database
spec:
  selector:
   matchLabels:
    app: mysql
    tier: database
  strategy:
   type: Recreate
  template:
   metadata:
    labels:
     app: mysql
     tier: database
   spec:
    containers:
     - image: mysql:5.7
       args:
        - "--ignore-db-dir=lost+found"
       name: mysql
       env:
        - name: MYSQL_ROOT_PASSWORD
          value: root
        - name: MYSQL_DATABASE
          value: easyfly_db

       ports:
        - containerPort: 3306
          name: mysql
       volumeMounts:
        - name: mysql-persistent-storage
          mountPath: /var/lib/mysql
    volumes:
     - name: mysql-persistent-storage
       persistentVolumeClaim:
```

```
        claimName: mysql-pv-claim
---

apiVersion: v1
kind: Service
metadata:
  name: mysql
  labels:
    app: mysql
    tier: database
spec:
  ports:
    - port: 3306
      targetPort: 3306
  selector:
    app: mysql
    tier: database
  clusterIP: None
```

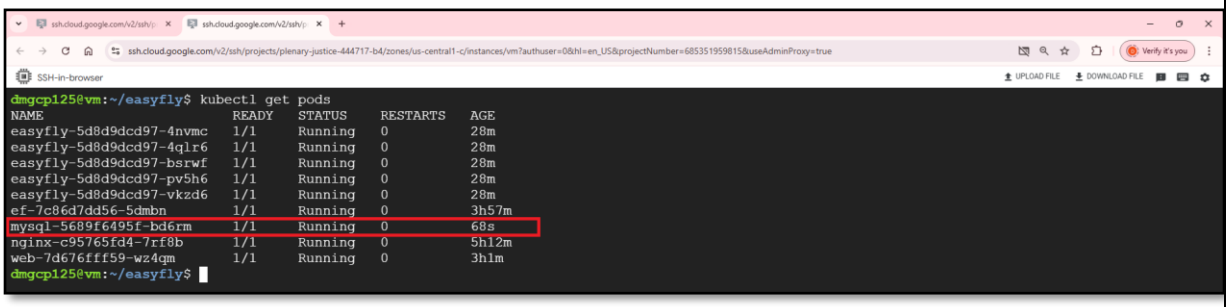
5

Create MySQL deployment

Command: kubectl apply -f db-deployment.yml

Check deployment

Command: kubectl get pods



Check database created inside MySQL

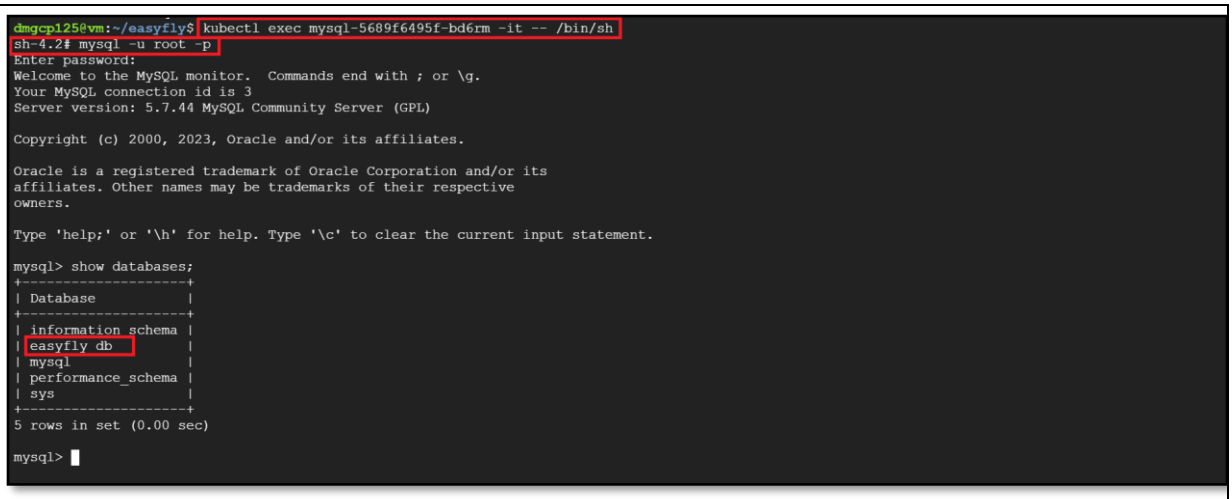
Command: kubectl exec <containername> -it -- /bin/sh

sh-4.2# mysql -u root -p

Enter password:root

Show databases;

Entet exit to come out

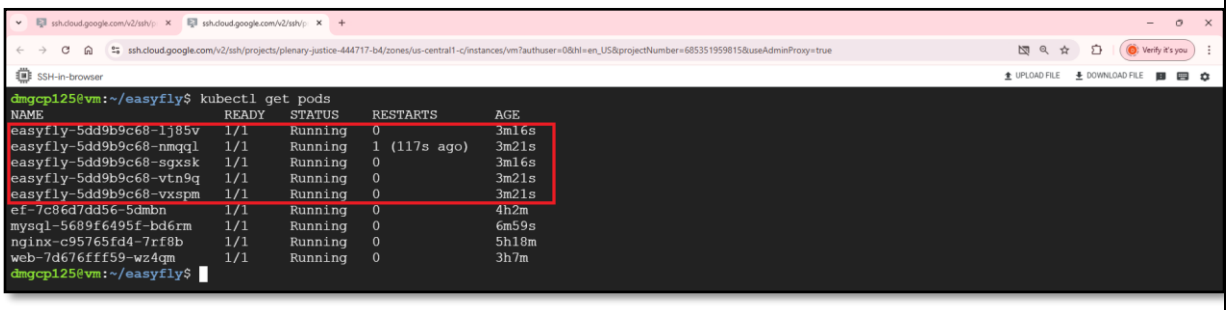


6 Deploy spring boot application

deployment.yml is as follows

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: easyfly
spec:
  selector:
    matchLabels:
      app: easyfly
  replicas: 2
  template:
    metadata:
      labels:
        app: easyfly
    spec:
      containers:
        - name: easyfly
          image: dadaramjadhav/easyfly:prod
          ports:
            - containerPort: 8081
---

apiVersion: v1
kind: Service
metadata:
  name: easyfly-svc
spec:
  selector:
    app: easyfly
  ports:
    - protocol: "TCP"
      port: 8081
      targetPort:
  type: LoadBalancer
```

	Deployment Command: kubectl apply -f deployment.yml  Get list of pods Command: kubectl get pods 
7	Access application  **Try Other endpoints** 1. curl 192.168.49.4:8081/api-v2/flights/ (GET) 2. curl 192.168.49.4:8081/api-v2/flights/3 (GET) 3. curl --header "Content-Type: application/json" --request POST --data '{ "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v2/flights/ (POST Request) 4. curl --header "Content-Type: application/json" --request PUT --data '{"flightId":6, "flightNumber":"JA-506","source":"Pune","destination":"Mumbai","status":"Running","duration":3,"fare":3000}' 192.168.49.4:8081/api-v2/flights/6 (PUT request) 5. curl --request DELETE 192.168.49.4:8081/api-v2/flights/6 (DELETE)

Activity 11: Deploying Spring Boot app with CPU and Memory Limits

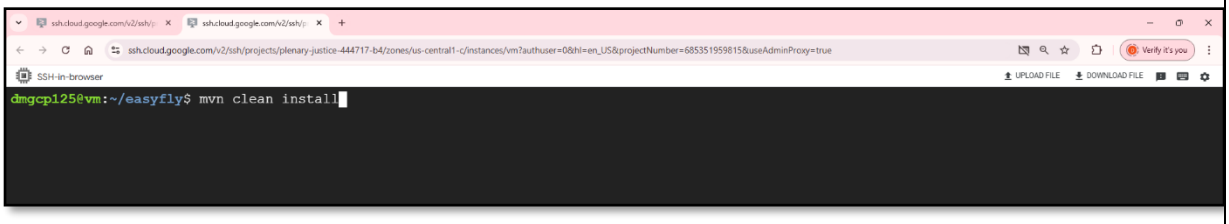
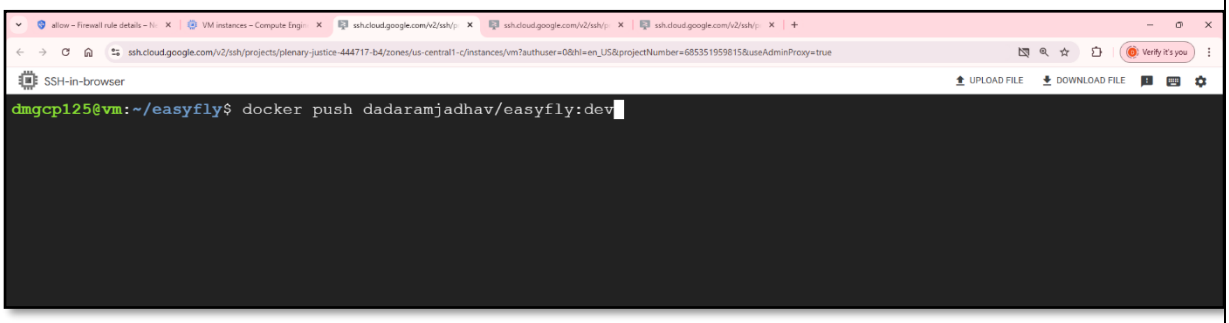
11.1. Prerequisite

Spring boot application

11.2. Scenario

Deploying spring boot application with CPU and memory limits

11.3. Steps

1	Go to vm
2	Build application Command: mvn clean install 
3	Build docker image Command: docker build -t dadaramjadhav/easyfly:dev -f Dockerfile-dev .  Push image to docker hub registry Command: docker push dadaramjadhav/easyfly:dev 
4	Refer below manifest file apiVersion: apps/v1 kind: Deployment metadata: name: easyfly

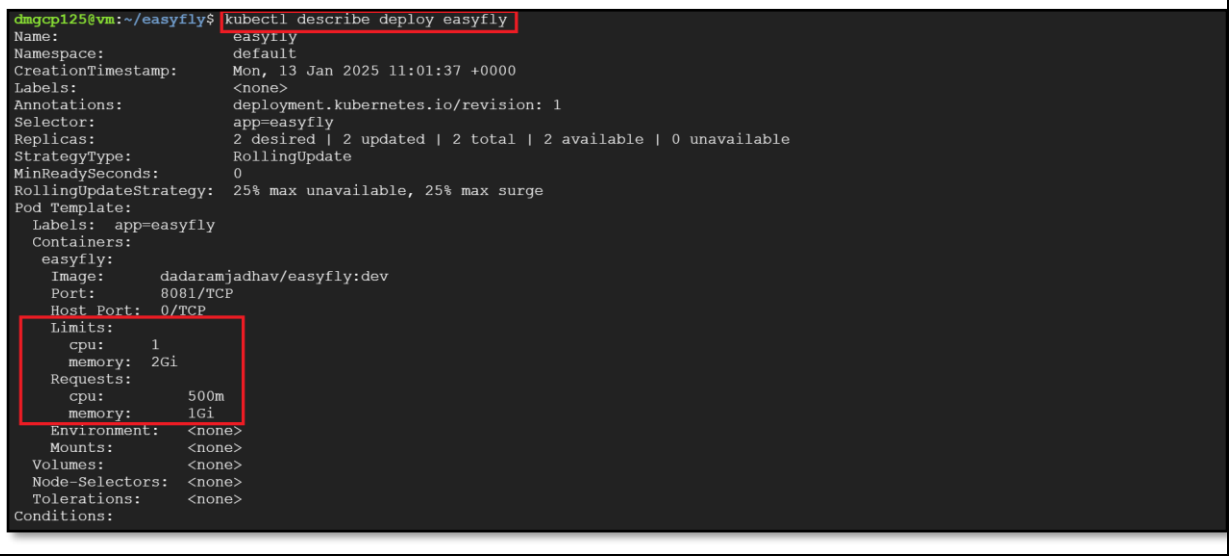
```
spec:
  selector:
    matchLabels:
      app: easyfly
  replicas: 2
  template:
    metadata:
      labels:
        app: easyfly
    spec:
      containers:
        - name: easyfly
          image: dadaramjadhav/easyfly:dev
          ports:
            - containerPort: 8081
          resources:
            requests:
              cpu: 500m
              memory: 1Gi
            limits:
              cpu: 1000m
              memory: 2Gi
---

apiVersion: v1
kind: Service
metadata:
  name: easyfly-svc
spec:
  selector:
    app: easyfly
  ports:
    - protocol: "TCP"
      port: 8081
      targetPort:
  type: LoadBalancer
```

5	Create deployment Command: kubectl apply -f deployment-with-cpu-memory-limits.yml 
6	Get deployment Command: kubectl get deploy

Describe deployment

Command: kubectl describe deploy <deployment-name>



-